

Pattern Recognition Lab Practice Sheet 2

Course Area	Pattern Recognition Lab
Question Type	Python Coding Practice
Total Questions	8
Focus	Model evaluation, classification metrics, confusion matrix, and interpretation
Purpose	Student practice using directly runnable sample code

Instruction: Run each code block directly in Python or Google Colab. The examples use sample actual labels, predicted labels, or probability values, so no trained model or test dataset is required for practice.

Question 1: Print Accuracy, Precision, Recall, and F1 Score

Question: After obtaining predictions from a binary classification model, write Python code to print Accuracy, Precision, Recall, and F1 Score.

Answer: Runnable Python Code

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Example actual labels
# 0 means Normal, 1 means Disease
y_true = [0, 1, 1, 0, 1, 0, 1, 0, 1, 0]

# Example predicted labels from a model
y_pred = [0, 1, 0, 0, 1, 0, 1, 1, 0, 0]

# Calculate performance metrics
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

# Print results
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

Expected Output

```
Accuracy: 0.7
Precision: 0.75
Recall: 0.6
F1 Score: 0.6666666666666665
```

Logic: This code compares actual labels with predicted labels. Accuracy gives the overall correct prediction rate. Precision, recall, and F1 Score give more detailed classification performance.

Question 2: Compute and Plot Confusion Matrix with TP and FN

Question: Write Python code to compute and plot the confusion matrix. Also print True Positive and False Negative values and interpret them for a disease detection dataset.

Answer: Runnable Python Code

```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Example actual labels
# 0 means Normal, 1 means Disease
y_true = [0, 1, 1, 0, 1, 0, 1, 0, 1, 0]

# Example predicted labels from a model
y_pred = [0, 1, 0, 0, 1, 0, 1, 1, 0, 0]

# Compute confusion matrix
cm = confusion_matrix(y_true, y_pred)

print("Confusion Matrix:")
print(cm)

# Extract TN, FP, FN, TP
TN, FP, FN, TP = cm.ravel()

print("True Negative:", TN)
print("False Positive:", FP)
print("False Negative:", FN)
print("True Positive:", TP)

# Plot confusion matrix
display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Normal", "Disease"]
)

display.plot()
plt.title("Confusion Matrix")
plt.show()

# Interpretation
print("TP: Patient has disease and model predicts disease.")
print("FN: Patient has disease but model predicts normal.")
```

Expected Output

```
Confusion Matrix:
[[4 1]
 [2 3]]
True Negative: 4
False Positive: 1
False Negative: 2
True Positive: 3
TP: Patient has disease and model predicts disease.
FN: Patient has disease but model predicts normal.
```

Logic: The confusion matrix shows correct and wrong predictions in table form. TP means correct disease detection. FN is risky because a diseased patient is predicted as normal.

Question 3: Display a Classification Report

Question: Write Python code to display a classification report showing precision, recall, F1 Score, and support for each class.

Answer: Runnable Python Code

```
from sklearn.metrics import classification_report

# Example actual and predicted labels
# 0 means Normal, 1 means Disease
y_true = [0, 1, 1, 0, 1, 0, 1, 0, 1, 0]
y_pred = [0, 1, 0, 0, 1, 0, 1, 1, 0, 0]

# Generate classification report
report = classification_report(
    y_true,
    y_pred,
    target_names=["Normal", "Disease"]
)

print(report)
```

Expected Output

	precision	recall	f1-score	support
Normal	0.67	0.80	0.73	5
Disease	0.75	0.60	0.67	5
accuracy			0.70	10
macro avg	0.71	0.70	0.70	10
weighted avg	0.71	0.70	0.70	10

Logic: The classification report gives class wise performance. It helps students see which class has better precision and recall. Support means the number of actual samples in each class.

Question 4: Calculate Metrics from TP, FP, FN, and TN

Question: Given TP = 35, FP = 5, FN = 10, and TN = 50, write Python code to calculate Accuracy, Precision, Recall, and F1 Score.

Answer: Runnable Python Code

```
# Given confusion matrix values
TP = 35
FP = 5
FN = 10
TN = 50

# Calculate metrics
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
f1 = 2 * (precision * recall) / (precision + recall)
```

```
# Print results
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

Expected Output

```
Accuracy: 0.85
Precision: 0.875
Recall: 0.7777777777777778
F1 Score: 0.823529411764706
```

Logic: This code practices manual metric calculation. TP, FP, FN, and TN are the main values used in classification evaluation. F1 Score balances precision and recall.

Question 5: Convert Predicted Probabilities into Class Labels

Question: A binary classifier gives probability outputs. Write Python code to convert probability values into class labels and then calculate performance metrics.

Answer: Runnable Python Code

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Actual labels
# 0 means Normal, 1 means Disease
y_true = [0, 1, 1, 0, 1, 0, 1, 0]

# Predicted probabilities from a sigmoid output
y_pred_prob = [0.10, 0.88, 0.40, 0.20, 0.91, 0.30, 0.75, 0.60]

# Convert probabilities to class labels using threshold 0.5
y_pred = []
for prob in y_pred_prob:
    if prob >= 0.5:
        y_pred.append(1)
    else:
        y_pred.append(0)

print("Predicted Labels:", y_pred)

# Calculate metrics
print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1 Score:", f1_score(y_true, y_pred))
```

Expected Output

```
Predicted Labels: [0, 1, 0, 0, 1, 0, 1, 1]
Accuracy: 0.75
Precision: 0.75
Recall: 0.75
```

F1 Score: 0.75

Logic: A sigmoid model gives values between 0 and 1. A threshold of 0.5 converts probabilities into class labels. After conversion, normal classification metrics can be calculated.

Question 6: Compare Two Models Using Accuracy and F1 Score

Question: Two binary classification models produced different predictions. Write Python code to compare the two models using Accuracy and F1 Score.

Answer: Runnable Python Code

```
from sklearn.metrics import accuracy_score, f1_score

# Actual labels
y_true = [0, 1, 1, 0, 1, 0, 1, 0, 1, 0]

# Predictions from two models
model_a_pred = [0, 1, 0, 0, 1, 0, 1, 1, 0, 0]
model_b_pred = [0, 1, 1, 0, 1, 0, 0, 0, 1, 0]

# Model A performance
model_a_accuracy = accuracy_score(y_true, model_a_pred)
model_a_f1 = f1_score(y_true, model_a_pred)

# Model B performance
model_b_accuracy = accuracy_score(y_true, model_b_pred)
model_b_f1 = f1_score(y_true, model_b_pred)

print("Model A Accuracy:", model_a_accuracy)
print("Model A F1 Score:", model_a_f1)

print("Model B Accuracy:", model_b_accuracy)
print("Model B F1 Score:", model_b_f1)

if model_b_f1 > model_a_f1:
    print("Model B performs better based on F1 Score.")
else:
    print("Model A performs better based on F1 Score.")
```

Expected Output

```
Model A Accuracy: 0.7
Model A F1 Score: 0.6666666666666665
Model B Accuracy: 0.9
Model B F1 Score: 0.8888888888888889
Model B performs better based on F1 Score.
```

Logic: This code compares two models using the same actual labels. F1 Score is useful when precision and recall are both important. The model with higher F1 Score is usually better for balanced classification.

Question 7: Plot a Normalized Confusion Matrix

Question: Write Python code to plot a normalized confusion matrix so that each row shows percentages instead of only counts.

Answer: Runnable Python Code

```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Example actual and predicted labels
y_true = [0, 1, 1, 0, 1, 0, 1, 0, 1, 0]
y_pred = [0, 1, 0, 0, 1, 0, 1, 1, 0, 0]

# Normalized confusion matrix
cm = confusion_matrix(y_true, y_pred, normalize="true")

print("Normalized Confusion Matrix:")
print(cm)

# Plot normalized confusion matrix
display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Normal", "Disease"]
)

display.plot(values_format=".2f")
plt.title("Normalized Confusion Matrix")
plt.show()
```

Expected Output

```
Normalized Confusion Matrix:
[[0.8 0.2]
 [0.4 0.6]]
```

Logic: A normalized confusion matrix shows rates instead of raw counts. Each row sums to 1. It helps compare performance when class sizes are different.

Question 8: Calculate False Negative Rate for Disease Detection

Question: In disease detection, False Negative is important. Write Python code to calculate False Negative Rate from actual and predicted labels.

Answer: Runnable Python Code

```
from sklearn.metrics import confusion_matrix

# Example actual and predicted labels
# 0 means Normal, 1 means Disease
y_true = [0, 1, 1, 0, 1, 0, 1, 0, 1, 0]
y_pred = [0, 1, 0, 0, 1, 0, 1, 1, 0, 0]

# Compute confusion matrix
cm = confusion_matrix(y_true, y_pred)
TN, FP, FN, TP = cm.ravel()

# Calculate False Negative Rate
false_negative_rate = FN / (FN + TP)
```

```
print("False Negative:", FN)
print("True Positive:", TP)
print("False Negative Rate:", false_negative_rate)

print("Interpretation:")
print("False Negative Rate shows the proportion of disease cases missed by the model.")
```

Expected Output

```
False Negative: 2
True Positive: 3
False Negative Rate: 0.4
Interpretation:
False Negative Rate shows the proportion of disease cases missed by the model.
```

Logic: False Negative Rate is important in medical diagnosis. A high value means many diseased patients are being missed. So, disease detection models should try to reduce FN.

Short Notes for Students

Term	Meaning
Accuracy	Overall correct prediction rate.
Precision	Out of predicted disease cases, how many are actually disease.
Recall	Out of actual disease cases, how many are correctly detected.
F1 Score	Balanced score between precision and recall.
Confusion Matrix	Table showing TN, FP, FN, and TP.
True Positive	Actual disease and predicted disease.
False Negative	Actual disease but predicted normal.
False Negative Rate	Proportion of actual disease cases missed by the model.